Day 11 — Jenkins Fundamentals

Status: ■ Complete

Objectives

- Understand Jenkins architecture: controller, agents, executors
- Install Jenkins (via apt and via Docker) and complete initial setup
- Navigate the Jenkins UI and understand core concepts
- Install and manage plugins
- Create your first Freestyle job

Lab Steps

Install Jenkins locally

- Install via apt on Ubuntu, unlock with `initialAdminPassword`
- Complete the setup wizard, install suggested plugins
- Create your first admin user

Install Jenkins via Docker (alternative)

- Run `jenkins/jenkins:lts` with a named volume for persistence
- Compare pros/cons vs a native install

Explore the UI

- Dashboard, Manage Jenkins, Build History, Blue Ocean (if installed)
- Global Tool Configuration (JDK, Git, Maven/Node versions)

Create a Freestyle Job

- New Item → Freestyle project named `hello-jenkins`
- Add a build step: `echo "Hello from Jenkins"`
- Run the build, inspect console output

Add an agent (optional, for HA understanding)

- Add a permanent agent node via SSH
- Restrict the `hello-jenkins` job to that node using labels

Key Takeaways

- Jenkins is controller/agent based — the controller should ideally not run builds directly in production
- Plugins extend nearly everything: SCM, notifications, build tools, security
- `JENKINS_HOME` holds all state — back it up

Checklist

- [x] Jenkins installed and accessible on port 8080
- [x] Initial admin user created
- [x] Suggested plugins installed
- [x] First Freestyle job created and run successfully

Day 11 — Interview Questions: Jenkins Fundamentals

What is Jenkins, and what problem does it solve in a software delivery pipeline?

Explain the controller/agent (master/slave) architecture. Why shouldn't builds run on the controller in production?

What is `JENKINS_HOME` and what critical data does it contain?

What's the difference between a Freestyle project and a Pipeline job?

How do you install a plugin, and how do you handle plugin dependency conflicts?

What is an executor, and how does it relate to concurrent builds?

How would you back up and restore a Jenkins instance?

What are the two ways to install Jenkins covered in this module, and what are the trade-offs?

How do you recover access if you forget the Jenkins admin password?

What's stored in the `secrets/` directory, and why is it critical to back up?

<details><summary>Answer notes</summary>

- Q2: Running builds on the controller risks resource contention and security exposure (build code runs with controller-level trust). Agents isolate build execution.
- Q7: Tar/backup `JENKINS_HOME` (config.xml, jobs/, secrets/, plugins/); test restores regularly, not just backups.
- Q10: `secrets/` holds encryption keys used to decrypt stored credentials — losing them makes existing encrypted credentials unrecoverable even with a config backup.

</details>